New Horizons®
Centro de Capacitación

Curso

Taller de IA con .NET

Docente: Ing. Edwin Fidel Barrientos Retuerto

Módulo 5:

Introducción a IA Generativa y Consumo de IA .NET

Objetivo del módulo: Diseñar e implementar agentes simples usando .NET

New Horizons®
Centro de Capacitación

Sesión 5:

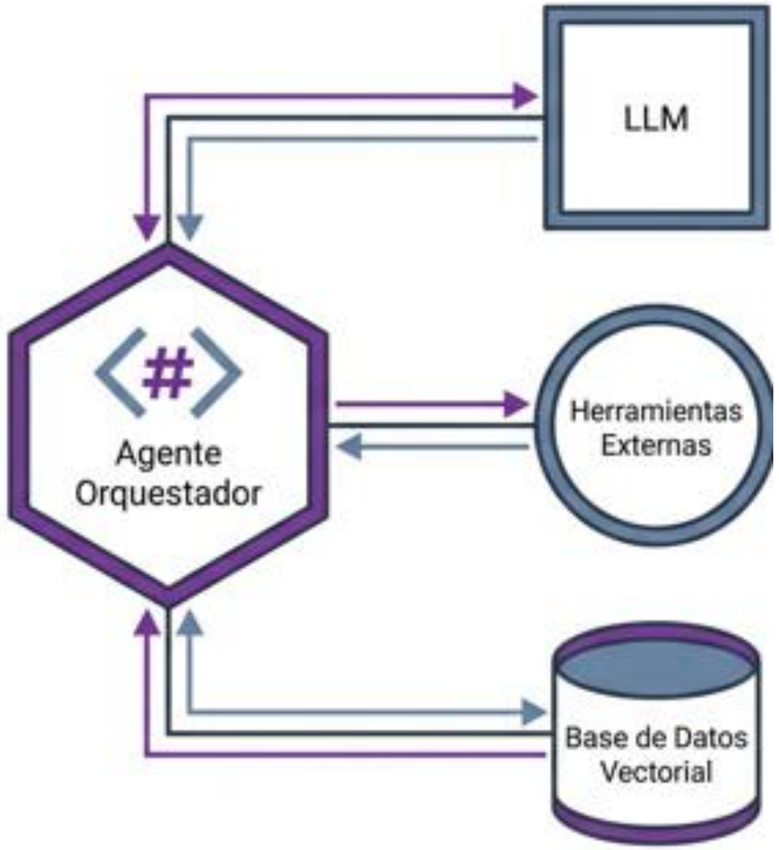
Diseño de Agentes en .NET

New Horizons®
Centro de Capacitación

Agenda

- Arquitectura básica de un agente
- Comunicación de Agente-Modelo
- Ejecución de Tareas Automáticas
- Ejercicio Practico

Arquitectura, Comunicación y Ejecución de Tareas en C#



La evolución hacia el software autónomo

New Horizons®
Centro de Capacitación

La integración de IA en entornos empresariales a evolucionado de simples llamadas a APIs estáticas hacia la orquestación de agentes autónomos. Éste documento explora los tres pilares fundamentales para su implementación en C#

1. Arquitectura Básica

Componentes, estado y despliegue (Local vs. Nube).

2. Comunicación

Interacción agente-modelo, streaming y SDKs.

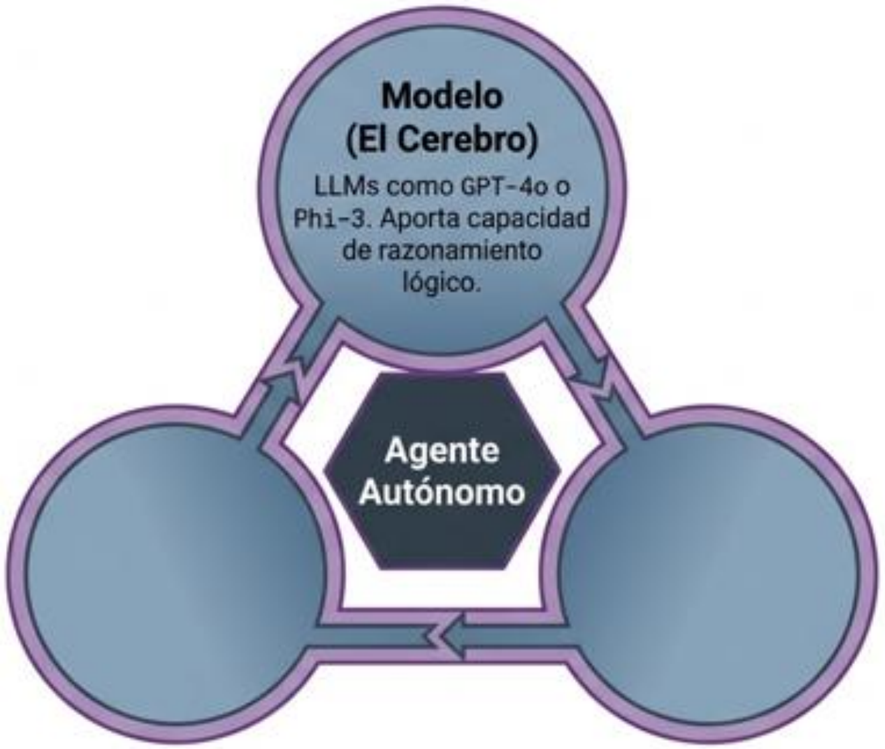
3. Automatización

Invocación de herramientas y automatización de navegadores.

Anatomia de un Agente IA

Un agente no es solo un modelo de lenguaje; es un sistema orquestado que combina capacidad de razonamiento con directivas claras y capacidades de acción.

Instrucciones (El comportamiento)
Reglas de negocio y personalidad que guían el objetivo del agente



Herramientas (Las Manos)
APIs, ejecución de código o recuperación de datos (ej. `CodeInterpreterToolDefinition`)

Gestión de Estado y Memoria Convencional

Los modelos de IA son inherentemente sin estado. El marco de agentes en C# gestiona el contexto a través de **Hilos (Threads) y Conversaciones,** persistiendo el historial para tareas de múltiples turnos.

Enfoque sin Estado

Las peticiones aisladas pierden el contexto anterior

Enfoque C# (PersistentAgentThread)

Orquestación Multi-Agente

Para problemas complejos, se componen flujos entre múltiples agentes especializados coordinados por herramientas de orquestación

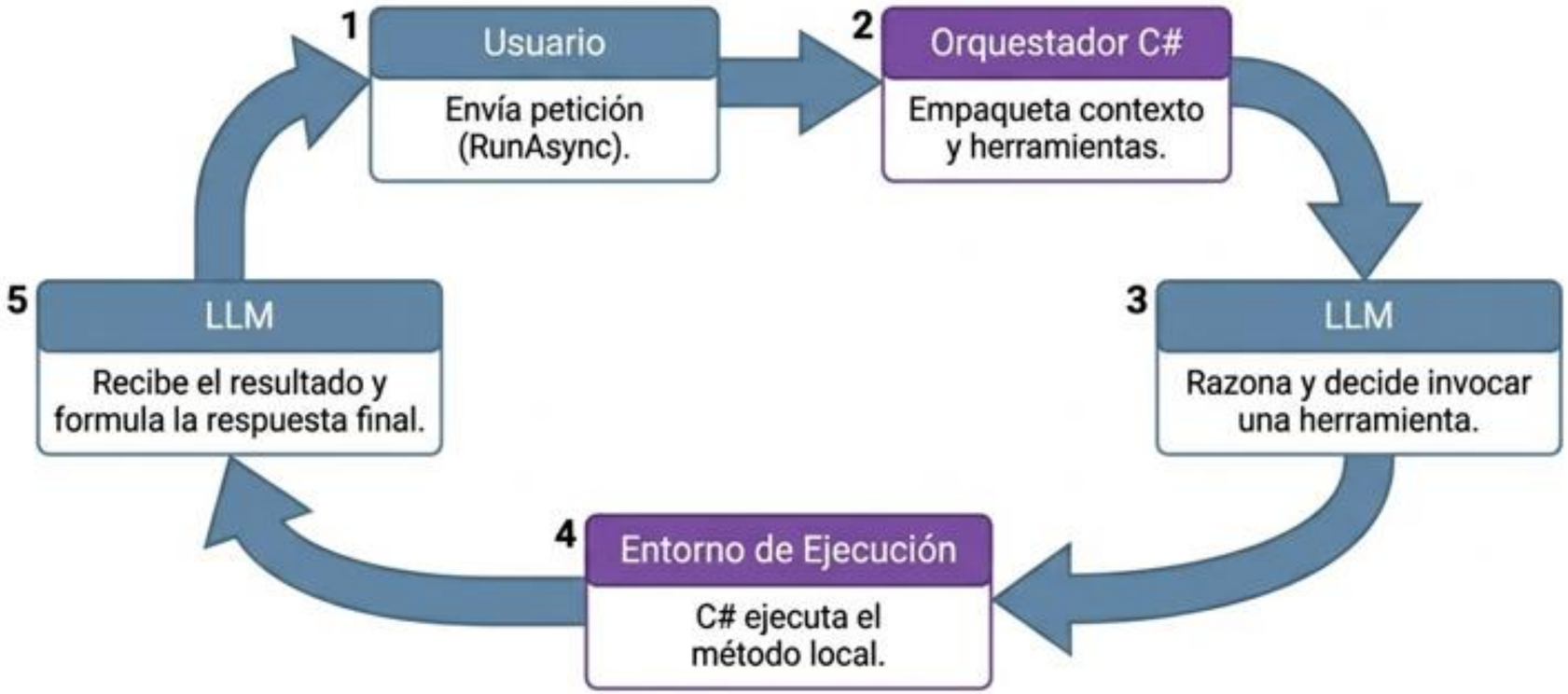
Comunicación entre agentes mediante intercambio de mensajes estructurados en un espacio de trabajo compartido.

Paradigmas de Despliegue: Nube vs Local

Nube (Azure AI Foundry)	Local (Foundry Local / LM Kit .NET)
Escala gestionada y alta disponibilidad	Privacidad total y ejecución offline (Soberanía de datos)
Modelos de Frontera (GPT-4o)	Modelos cuantizados optimizados para CPU/GPU/NPU (Phi3-Qwen)
Integración nativa con Azure Entra ID y seguridad perimetral	Integración directa en la aplicación

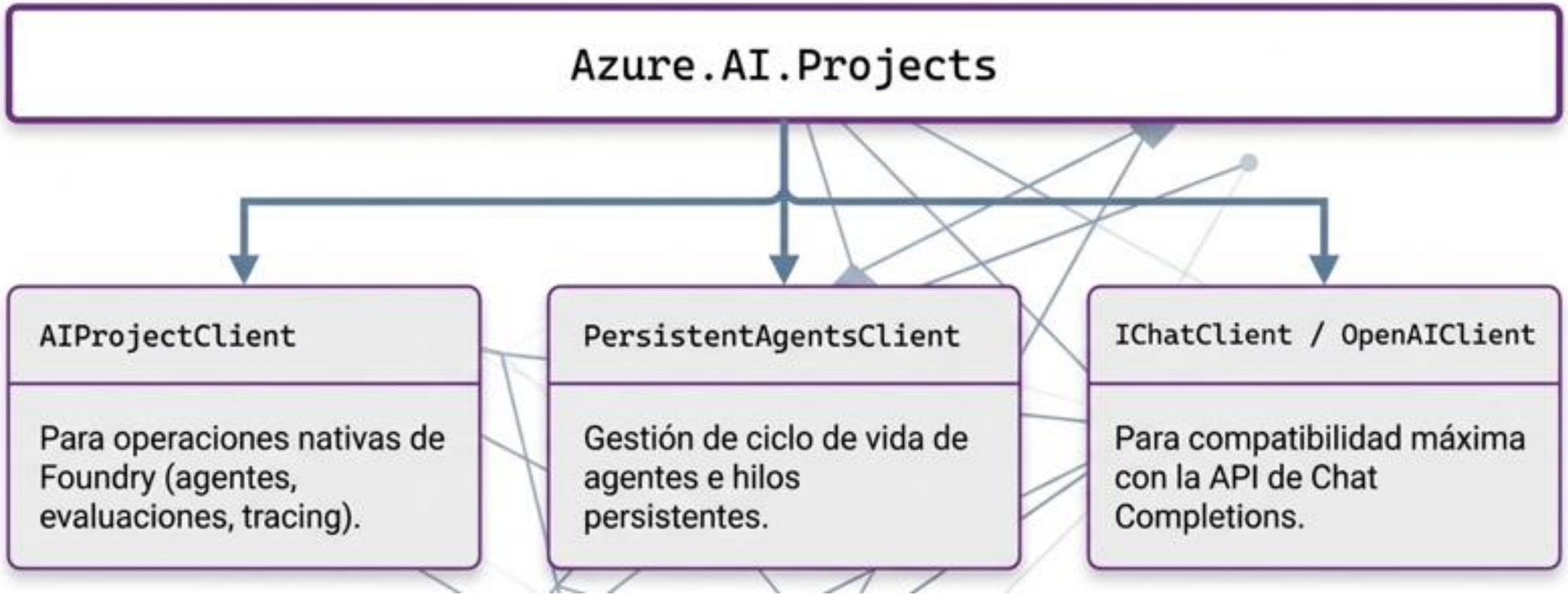
El ciclo de vida de la Interacción (Data Flow)

La comunicación Agente-Modelo es un bucle asíncrono donde el modelo decide de forma autónoma cuándo necesita información externa.



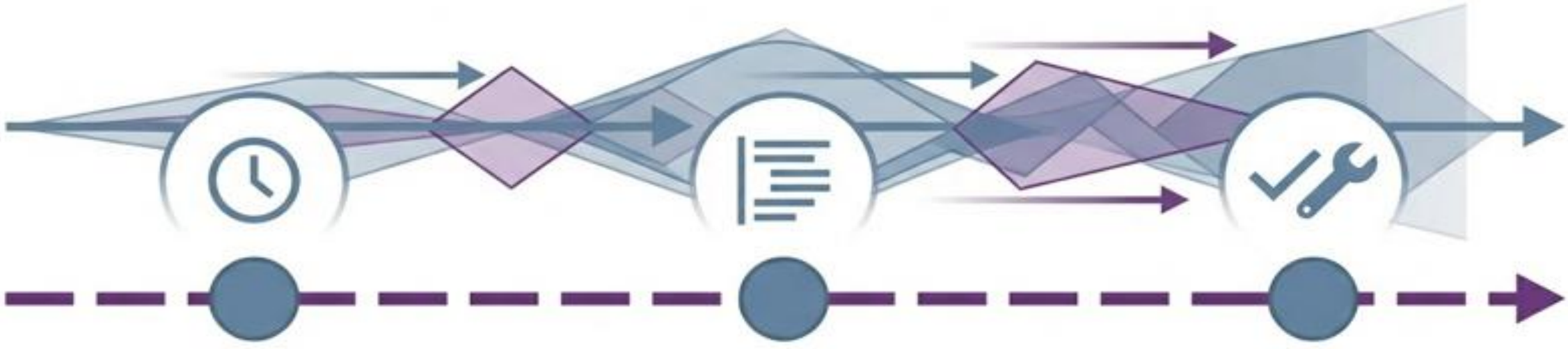
Endpoints y SDK Principales en C#

.NET proporciona abstracciones unificadas para interactuar tanto como Foundry como con modelo directos de OpenAI, garantizando compatibilidad de interfaces.



Streaming y Observabilidad en Tiempo Real

Debido a que la automatización puede requerir tiempos prolongados, el uso de flujos de datos asíncronos (IAsyncEnumerable) es crítico para la experiencia del usuario y la telemetría.



`response.created`

Inicio de la conexión

`response.output_text.delta`

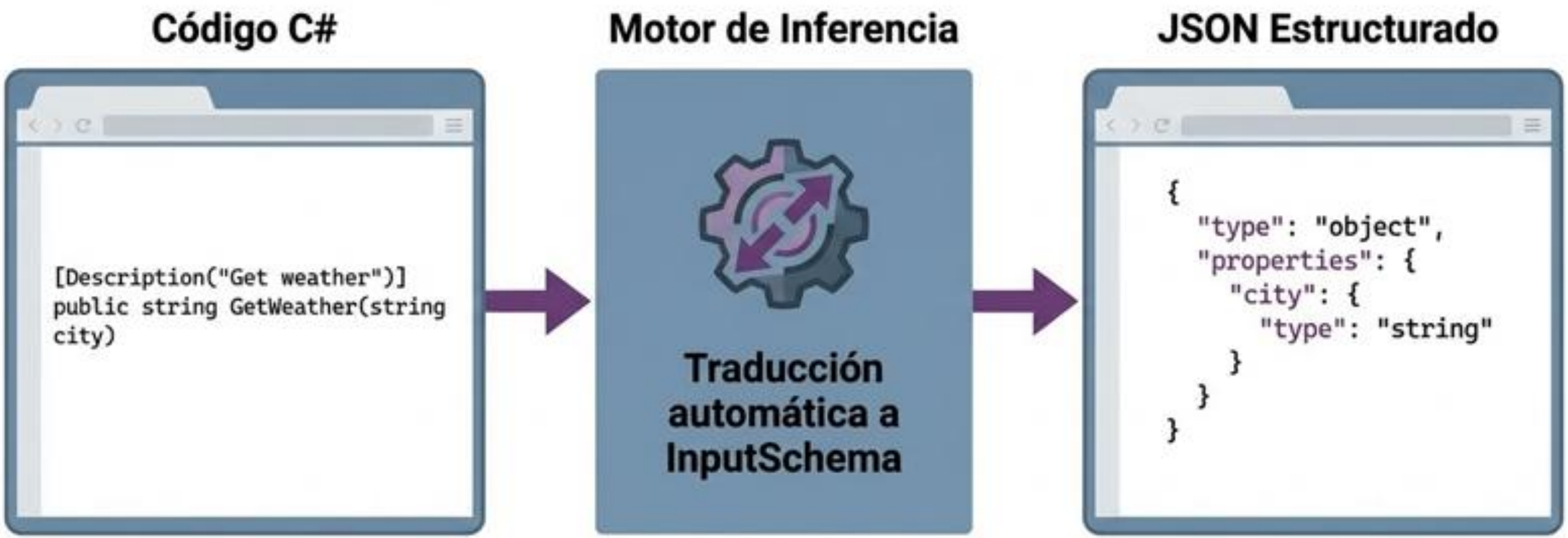
Fragmentos de texto Procesados en tiempo real

`response.output_item.done`

Confirmación de llamada a Herramienta (ej. `Browser_automation_review_call`)

Innovación de Herramientas (Function Calling)

El puente entre el lenguaje natural y el código C#. Las funciones se exponen como esquemas JSON que el modelo interpreta para generar argumentos válidos.



Capacidades de Ejecución Automática

Los agentes pueden interactuar con el mundo exterior mediante conectores especializados, transformando intenciones en acciones computacionales.

1. Recuperación de Datos

APIs REST (ej. `WeatherTool`, `CurrencyTool`), bases de datos vectoriales (`Azure AI Search`).

2. Ejecución de Código

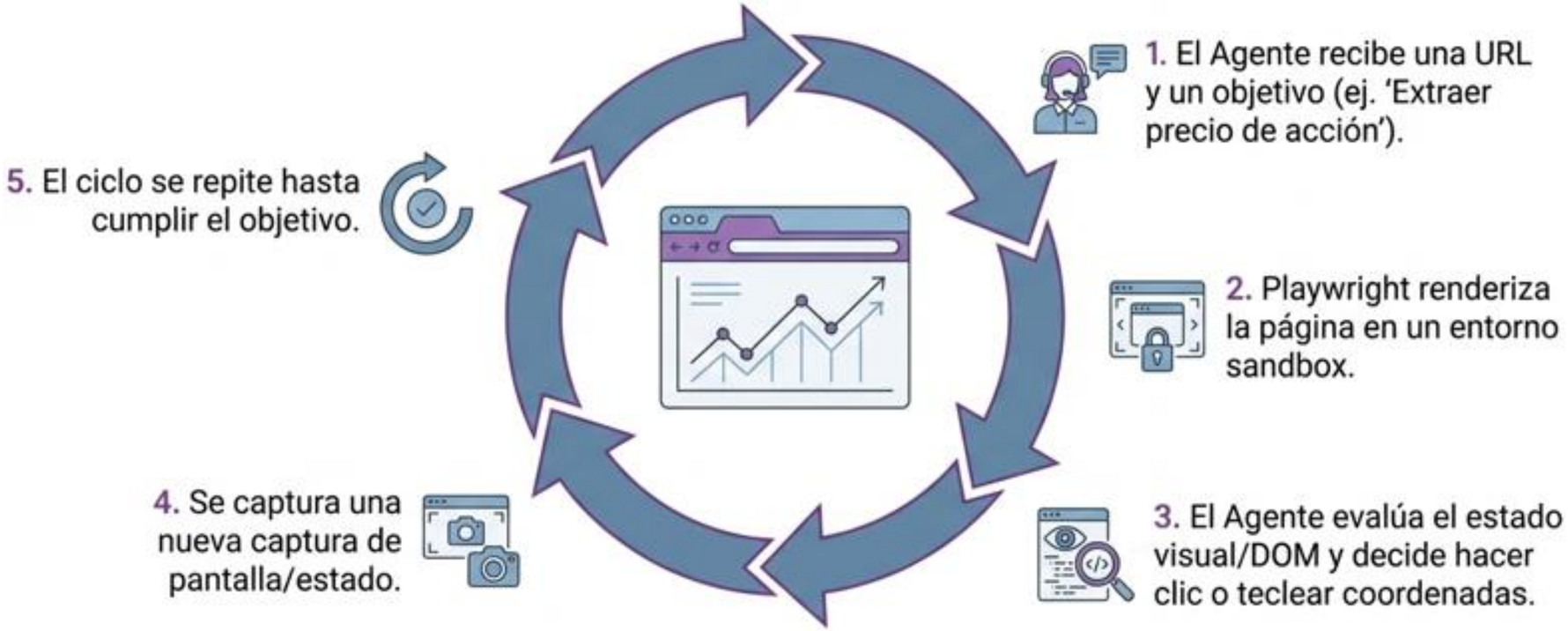
Herramientas como `CodeInterpreterToolDefinition` para resolver matemáticas, visualizar datos o manipular archivos en un sandbox.

3. Interacción con Sistemas

Herramientas CRUD web (`Create`, `Read`, `Update`, `Delete`) expuestas vía especificaciones `OpenAPI`.

Automatización de navegadores con playwright

La herramienta BrowserAutomationPreviewTool crea sesiones aisladas donde el modelo analiza HTML/DOM y coordina acciones de navegación.



Implementación Práctica en C#

New Horizons®
Centro de Capacitación

Configuración del cliente, inyección de herramientas de automatización y bucle de procesamiento de respuestas requeridas

Paso 1: Inicialización de AIProjectClient con credenciales gestionadas.

```
var client = new AIProjectClient(
    ...[redacted],
    ...[redacted]
);
```

Paso 2: Registro de BrowserAutomationPreviewTool asociando el Connection ID de Playwright.

```
client.RegisterTool(new BrowserAutomationPreviewTool(playwrightConnectionId))
...
```

Paso 3: Definición de PromptAgentDefinition (asignando el modelo y la herramienta).

```
var agent = new PromptAgentDefinition(model, tool)
    ...[redacted],
    ...[redacted]
);
```

Paso 4: Bucle CreateResponseStreaming exigiendo el uso de herramientas (ToolChoice.Required).

```
await foreach(var response in agent.CreateResponseStreaming(…, ToolChoice.Required))
{
    // [redacted]
    ...
}
```

Consideraciones de Seguridad y Producción

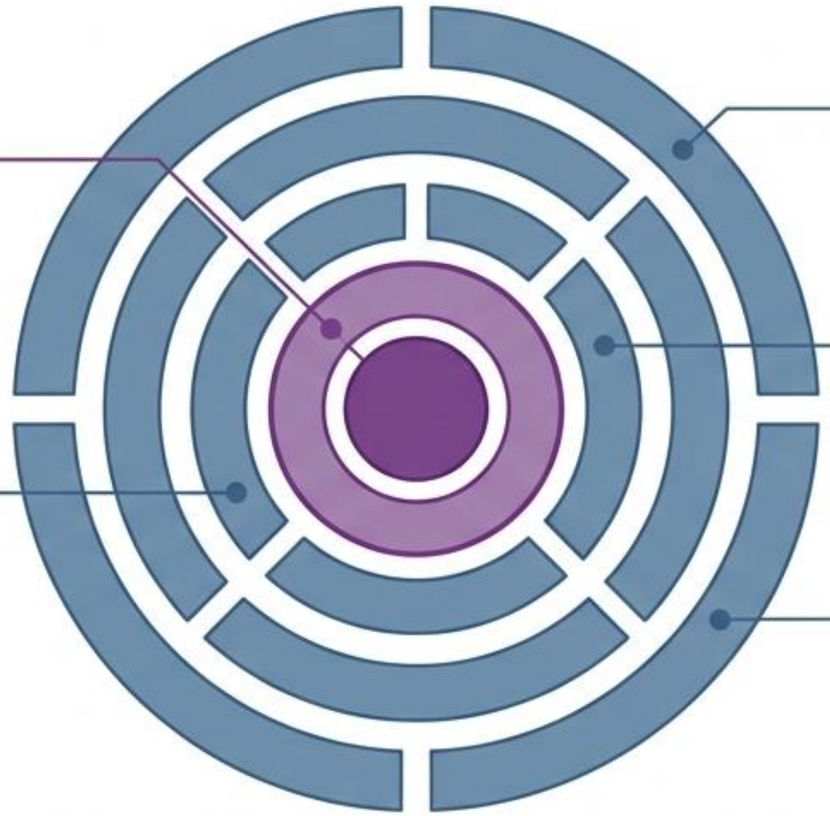
Otorgar autonomía a sistemas IA introduce riesgos significativos. La arquitectura debe aislar la ejecución y asegurar el principio de mínimo privilegio

Sandboxing

Ejecución de código y navegadores (Playwright) en máquinas virtuales de bajo privilegio.

Identidades Gestionadas (IAM)

Uso de Azure Entra ID (ej. rol Azure AI User) en lugar de claves estáticas incrustadas.



Filtros de Contenido

Tiempos de espera estrictos (Timeouts) y validación de esquemas bucles infinitos.

Circuit Breakers

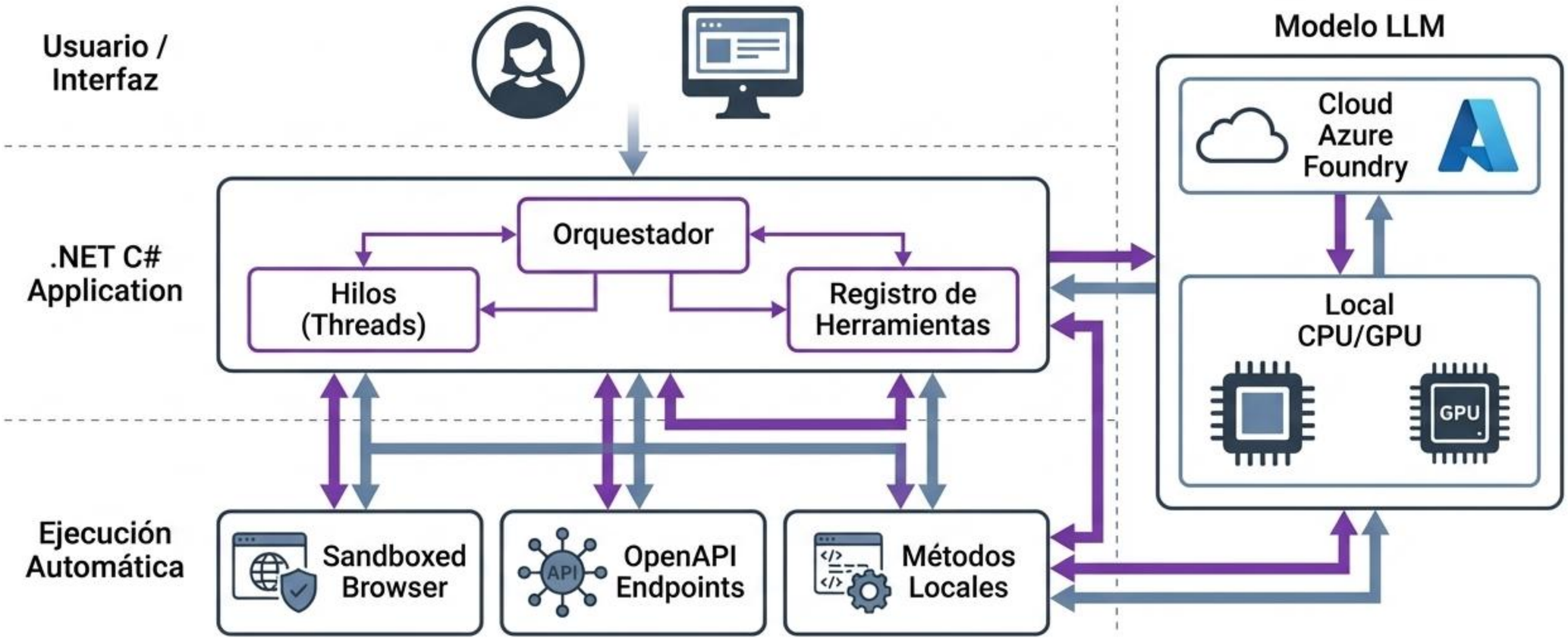
Tiempos de espera estrictos (Timeouts) y validación de esquemas JSON para evitar bucles infinitos.

Filtros de Contenido

Moderación integrada para sanitizar entradas y salidas.

Arquitectura de Referencia: .NET AI Agent

La orquestación del C# permite combinar la robustez del desarrollo empresarial con el razonamiento dinámico de la Inteligencia Artificial



New Horizons
Centro de Capacitación

Conclusiones

- **Microsoft Foundry unifica el desarrollo de IA** a nivel empresarial Microsoft Foundry se presenta como una plataforma unificada (disponible tanto en la nube como en local) que combina infraestructura de grado de producción con interfaces amigables

- **El ecosistema .NET/C# está fuertemente respaldado para la IA** Existe un soporte robusto para construir aplicaciones de IA dentro del ecosistema de Microsoft utilizando C# y .NET. Herramientas como el SDK de Foundry Local en C#, el marco de agentes de Microsoft, y librerías especializadas como LlmTornado y LM-Kit.NET, permiten a los desarrolladores integrar modelos de lenguaje (LLMs) directamente en sus aplicaciones existentes con un alto nivel de control

- **Las "Herramientas" (Tools) son esenciales para la autonomía de los agentes** Los agentes de IA dejan de ser simples chatbots gracias a su capacidad para invocar herramientas externas. Estas herramientas permiten a los agentes realizar acciones del mundo real, como buscar en la web, ejecutar código, consumir APIs, realizar conversiones e incluso automatizar tareas complejas en navegadores web utilizando herramientas como Playwright

- **La seguridad, el control y la privacidad son prioridades críticas** Al implementar agentes de IA en entornos corporativos, la seguridad no es opcional. El uso de herramientas de IA conlleva riesgos (como la ejecución de comandos no deseados), por lo que las plataformas exigen el uso de controles de acceso basados en roles (RBAC), filtros de seguridad de contenido, ejecución en entornos aislados (sandboxes) e integraciones con Microsoft Entra ID para proteger sistemas e información sensible

- **La orquestación multiagente resuelve problemas complejos** Para tareas avanzadas, la tendencia es utilizar la orquestación de múltiples agentes, donde diferentes agentes especializados (por ejemplo, un investigador, un analista y un redactor) colaboran para lograr un objetivo común

GRACIAS